CogniLink: A Low-Cost Wearable Motion-Tracking Human-Machine Interface and Bilingual Augmentative Communication Dashboard for Severe Motor Impairment

Abstract—People with severe motor impairment often cannot speak or use a standard mouse, and the devices that give them access to augmentative and alternative communication (AAC) are costly and rarely support languages such as Bengali. This paper presents CogniLink, an open-source system that pairs a wearable inertial pointer with a bilingual Bengali and English communicator. A commodity module, worn on a finger or at four other body sites, corrects its gyroscope for bias, applies a velocity control law with threshold subtraction and saturation, and appears to phones, tablets, and computers as a standard Bluetooth Low Energy mouse, so no driver is needed. The same module serves a web communicator with English and Bengali keyboards, the latter ordered by place of articulation and forming conjuncts through the hasanta, together with dwell selection, eight bilingual emergency phrases, and speech with an offline fallback. The hardware costs US$15. Bench tests with three units and four hosts measured a motion-to-photon latency of 44.35 ± 4.36 ms on a 60 Hz television, which a stage model attributes mainly to the Bluetooth connection interval and the display. With the deadzone enabled, the cursor did not move during 60 s at rest, against 25.4±4.2 px of drift without it. Dwell selections fired 2.41 ± 4.13 ms after their nominal times, offline speech started after 43.6 ms in English and 58.3 ms in Bengali, and the pointer operated two third-party web communicators on Windows, macOS, Android, and Linux. These results set a technical baseline for clinical evaluation.

Index Terms—Assistive technology, augmentative and alternative communication, Bengali orthography, Bluetooth Low Energy, dwell selection, human-machine interface, inertial measurement unit, microcontroller firmware, motion-to-photon latency, vertical synchronization, wearable sensors.

I. Introduction

PEOPLE who lose speech and most voluntary movement through amyotrophic lateral sclerosis (ALS), cerebral palsy (CP), high cervical spinal cord injury at the C3 to C5 levels, or brainstem stroke often keep the ability to understand others and to decide what they want to say. In ALS, dysarthria is the first or main sign in 25% to 30% of patients and affects more than 80% overall [1]. In a Norwegian registry study of 564 children with CP, about one child in five had no verbal speech [2]. Augmentative and alternative communication (AAC) can give these people a voice, but only if they can reach the letters or phrases on a screen. Even with direct access through a mouse, touchscreen, or eye tracker, users of text-based AAC typically produce 8 to 10 words per minute, and switch scanning averages about 1.7, while conversational speech runs at 120 to 185 [3]. The access method therefore sets a firm limit on how quickly and how independently a person can communicate.

Access is harder still in low- and middle-income countries. The World Health Organization and UNICEF estimate that more than 2.5 billion people need at least one assistive product and that access ranges from 3% in some countries to 90% in others [4]. In a household survey in two districts of Bangladesh, 71% of people with a functional difficulty reported an unmet need for assistive products [5]. Dedicated head mice and eye trackers cost from about US$600 to more than US$1,200 (Table I), before the computer that runs the communication software. Language is a second barrier, because work on AAC in languages other than English remains limited [6], even for languages with very large populations. Bengali, for example, has about 273 million speakers and ranks seventh among the world's languages by total number of speakers [7]. For a Bengali-speaking family, a usable system has to be affordable, run on devices the family already owns, speak their language, and be repairable with parts that can be bought locally.

Several of the necessary pieces already exist. Wearable head mice translate head rotation into cursor movement [8], [9], open-source alternative mice present themselves to the host as standard input devices [10], [11], and free web communicators offer multilingual boards and hover selection [12], [13]. What is still missing is a low-cost wearable pointer that needs no driver, paired with a Bengali communicator built around dwell selection and urgent needs, and characterized in enough technical detail that clinicians and engineers can judge whether it is ready for testing with patients.

This paper presents CogniLink, a communication system built from one commodity module and a web application. A unit worn on a finger, or on the wrist, an eyeglass temple, a headband, or the big toe (Fig. 1), reads angular velocity from its inertial measurement unit (IMU) and moves the cursor of a phone, tablet, or computer, which recognizes it as an ordinary Bluetooth Low Energy (BLE) mouse. The same unit serves the communicator as a web page over Wi-Fi, so the host needs nothing more than a browser. The communicator provides an English keyboard, a Bengali keyboard that follows the traditional order of the alphabet, dwell selection with four dwell times, eight bilingual emergency phrases, and speech output with an offline fallback. The hardware costs US$15.

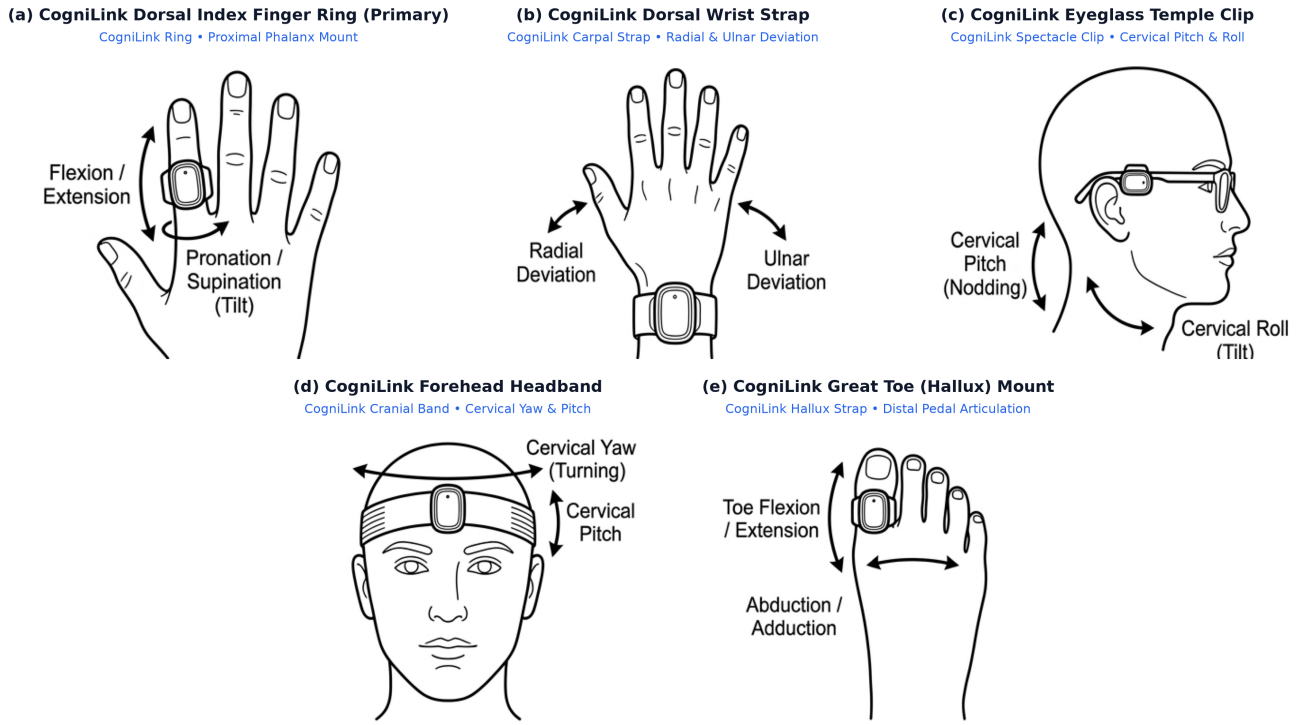


Fig. 1. Mounting configurations supported by the CogniLink cradle: a dorsal finger ring on the proximal phalanx (default), a wrist strap, an eyeglass temple, a headband, and the dorsum of the big toe (hallux).

This paper reports the design, implementation, and bench characterization of the system. Evaluation with patients, caregivers, and clinicians is the next stage of the project and will be reported separately. The paper makes four contributions:

1) A driver-free wearable inertial pointer built on a single commodity module, with a velocity control law that combines bias correction, threshold subtraction, and saturation, and an analysis of its onset, gain, and saturation (Sections IV and V).
2) A bilingual communicator served by the wearable itself, with a Bengali keyboard that forms conjuncts through the hasanta, dwell selection, and a tray of emergency phrases (Section VI).
3) A bench characterization on four host platforms covering motion-to-photon latency, with a stage-by-stage latency model, resting stability, report timing, dwell timing, power consumption, speech onset, and compatibility with third-party web AAC (Sections VII and VIII).
4) An open release of the hardware design, firmware, communicator, and benchmark data [14].

Section II reviews related work, and Section III states the design goals. Sections IV to VI describe the hardware, firmware, and communicator. Sections VII and VIII present the evaluation methods and results, Section IX discusses their implications and the limitations of the system, and Section X concludes.

II. Related Work

A. AAC and Access Methods

High-tech AAC pairs a selection interface with speech output, and the access method largely sets the pace of communication. Direct selection is faster than switch scanning, but both are far slower than speech [3]. Eye trackers such as the PCEye 5 give gaze-only control of a Windows computer at an estimated cost of US$1,249 [15]. Web-based AAC has lowered the software barrier. Cboard, an open-source communication board, lists Bengali among its languages and can run and speak offline in Chrome [12], and AsTeRICS Grid, a free grid communicator, accepts touch, click, hover, and scanning input and keeps working offline once loaded [13]. These tools solve much of the software problem, but they assume that the user already has a reliable way to point or to press a switch.

B. Motion-Based and Wearable Pointing

When hand function is limited, cursor control is often moved to the head or face. Camera Mouse tracks a facial feature through a webcam [16] and is free to use [17]. In an ISO 9241-9 evaluation it reached 0.65 bits/s with dwell selection, against 2.30 bits/s for a touchpad with tapping, and cursor control was erratic, with frequent target re-entries [18]. Head tracking with a tablet's front camera reached 1.42 bits/s on the multi-directional task of ISO 9241-411 [19]. Cameras also depend on lighting and on where the user sits [20]. Vojtech et al. combined head-tilt accelerometry with facial surface electromyography (EMG); with 24 non-disabled adults this gave better

TABLE I
CogniLink Compared With Existing Access Devices and AAC Software. Costs Are Published Estimates in US Dollars; n/a Means Not Applicable and n/r Means Not Reported.

Solution	Type	Input	Selection	Open	Bengali	Cost (USD)
PCEye 5 [15]	Eye tracker	Eye gaze	Gaze software	No	n/a	1,249
Quha Zono [8]	Head mouse	Head or other body site	Dwell software or switch	No	n/a	999
GlassOuse v1.4 [9]	Gyroscopic mouse	Head	0.5 s dwell or switch	No	n/a	599
Camera Mouse [16], [17]	Camera pointer	Face via webcam	Dwell	n/r	n/a	Free
LipSync [10]	Mouth joystick	Mouth	Sip and puff	Yes	n/a	Under 300 (build)
FLipMouse [11]	Joystick	Lips or finger	Sip, puff, or switch	Yes	n/a	n/r
Cboard [12]	Web AAC	Touch or pointer	Click or touch	Yes	Yes	Free (paid tier)
AsTeRICS Grid [13]	Web AAC	Touch, pointer, or switch	Click, hover, or scan	Yes	n/r	Free
CogniLink (this work)	Pointer and web AAC	Finger, wrist, head, or toe	Dwell or button	Yes	Yes	15

accuracy and path efficiency than Camera Mouse, which was faster [17], and Groll et al. compared click methods for the same kind of accelerometer-based cursor [21]. Auxilio, developed in Bangladesh, uses a head-mounted IMU to point and infrared sensing of cheek twitches to click, and it communicates with the computer through a receiver and driver software. With 10 participants without motor impairments it scored 78.75 on the System Usability Scale, against 48.75 for a commercial camera-based mouse [20], and its acceptability has been studied with people with upper-limb disability [22]. Commercial wearable head mice such as the Quha Zono [8] and the gyroscopic GlassOuse [9] follow the same principle at prices of several hundred US dollars or more. Among open designs, LipSync is a mouth-operated joystick with sip-and-puff clicking [10], and FLipMouse behaves as a standard mouse, keyboard, or joystick without extra software [11].

Finger-worn IMUs have received less attention as accessibility inputs. MouseRing tracks a finger sliding on a physical surface with one or two IMU rings and came close to touchpad speed in a Fitts' law study with 12 participants [23], but it needs a surface and deliberate finger sliding. Mouse on a Ring instead maps the tilt and acceleration of a ring moved in the air to cursor motion [24]. We are not aware of a finger-worn in-air pointer that has been combined with dwell-based AAC or with Bengali.

C. Dwell Selection

Dwell selection lets a person click by holding the pointer still. It removes the need for a switch but invites unintended selections whenever the pointer rests on a target, the so-called Midas touch problem [25]. Most eye-typing studies have used constant dwell times of 450 to 1000 ms and report 5 to 10 words per minute [26]. Helmert et al. obtained their best overall gaze-typing performance with a 500 ms dwell [27], and novices who were allowed to adjust the dwell shortened it from 876 to 282 ms over ten sessions while their speed rose from 6.9 to 19.9 words per minute [26]. Dynamic schemes shorten the dwell for likely keys. Cascading dwell reached 12.39 words per minute against 10.62 for a static dwell with 17 non-disabled participants, and 9.51 words per minute with five people with ALS [28]. A recent meta-analysis reports 500 to 600 ms as a common balance between speed and accuracy in eye tracking [29]. For motion-based pointing the evidence is thinner: Groll et al. tested dwell times of 0.5, 1.0, and 1.5 s with a head-tilt accelerometer cursor [21]. Gaze reaches a target almost at once, whereas a body-worn pointer has to be steered onto it, so dwell times from eye typing may not transfer directly.

D. Text Entry Rate and Word Prediction

Word and phrase prediction is the most common way to raise AAC text entry rates [3], but its benefit depends on the user. Koester and Levine first modeled and then measured this trade-off. The time needed to read and judge a prediction list can offset the keystrokes it saves [30], and for users with spinal cord injury, word prediction significantly lowered the text entry rate, while it raised the rate only modestly for able-bodied users [31]. For a new language and a new input device, these findings argue for measuring the cost of prediction rather than assuming a gain.

E. Bengali Text Entry and Multilingual AAC

Bengali is written in an abugida. Each consonant carries an inherent vowel, dependent vowel signs (kar) attach to consonants, and consonant clusters are written as conjuncts by placing the hasanta (virama, U+09CD) between consonants, which the text renderer then joins into one glyph. One visible character can therefore take several key selections, and the same text can be entered in different ways. Sarcar et al. showed that text entry metrics designed for English do not transfer directly to Bengali for this reason and proposed Bengali-specific definitions [32], and Arif and Fardeen built a phrase set for Bengali text entry evaluations from real text messages [33]. Among the communicators reviewed above, Cboard supports Bengali [12], but we are not aware of a Bengali keyboard designed for dwell selection with a body-worn pointer.

III. Design Goals

From the work reviewed above and the conditions CogniLink is meant for, we set six design goals.

- G1 Use whatever movement remains. Controllable movement differs between people and may decline over time, so the unit can be worn on a finger, wrist, eyeglass temple, headband, or toe, and the dwell time is chosen per user.
- G2 Run on devices people already own. The pointer should work with phones, tablets, and computers, including computers that drive a television at the bedside, without drivers or apps, as standard input devices such as FLipMouse already do [11]. The communicator should need only a browser.
- G3 Speak Bengali and English. Both languages should be available for typing and for speech, since AAC outside English remains scarce [6], and speech should keep working without an internet connection.
- G4 Put urgent needs first. Requests about suction, pain, breathing, or repositioning should be one selection away, because composing text letter by letter is slow [3].
- G5 Keep cost and repair within local means. The hardware should use commodity parts and an open design, given how limited access to assistive products is in low-resource settings [4], [5].
- G6 Tolerate drift and small involuntary movement. Sensor drift and small unintended movements should neither move the cursor nor trigger selections, a known weakness of dwell-based pointing [18], [25].

IV. Hardware

A. System Overview

Fig. 2 shows the architecture. The wearable unit maintains two independent links. Over BLE it acts as a standard mouse: gyroscope readings are corrected for bias, passed through a deadzone and a gain, clipped to the range of a mouse report, and sent to the host, whose operating system moves the cursor. Over Wi-Fi it serves the communicator as a single web page and pushes telemetry, including the battery level, to that page. The communicator runs in the host's browser, where it handles dwell selection, message composition, and speech. Because the pointer is an ordinary mouse, it is not tied to the communicator and can also operate other software, including third-party AAC.

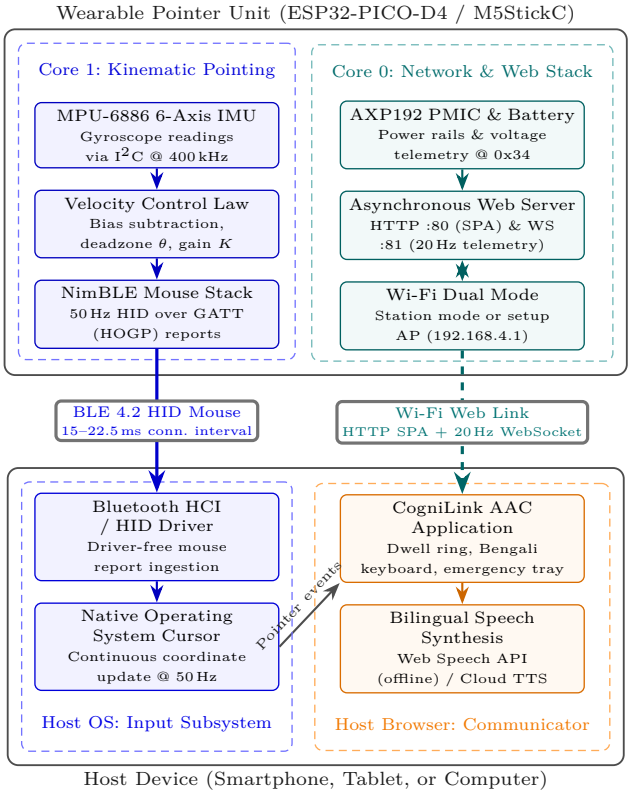


Fig. 2. System architecture and inter-device communication model. In the wearable unit, Core 1 executes the 50 Hz kinematic pointing pipeline (MPU-6886 sampling, bias correction, non-linear velocity control law, and NimBLE HID mouse reporting). Core 0 runs the power and web subsystem (AXP192 telemetry, asynchronous HTTP server, and 20 Hz WebSocket broadcast over Wi-Fi). On the host device, the OS input stack receives BLE reports to steer the native cursor, while the browser executes the self-hosted bilingual AAC communicator with dwell selection and multi-engine speech synthesis.

B. Form Factor and Mounting Sites

The default configuration is a ring module seated on the dorsal side of the proximal phalanx of the dominant index or middle finger (Fig. 1). An M5StickC module [34] sits in a cradle printed in-house by fused deposition modeling (FDM) from polyethylene terephthalate glycol (PETG) and is held by a silicone strap with an adjustable hook-and-loop fastener. The module measures 48.2 × 25.5 × 13.7 mm and weighs 15.1 g including its battery [34]. On the finger, the unit senses flexion, extension, and rotation of the finger and hand.

For people who cannot use a finger, the same cradle attaches at four other sites (Fig. 1). A dorsal wrist strap captures radial and ulnar deviation for people with severe hand spasticity or finger contractures who keep wrist movement. A clip on an eyeglass temple turns head pitch and roll into vertical and horizontal cursor movement for people with high cervical spinal cord injury who keep voluntary head movement. An elastic headband serves people who keep some neck rotation, for example in advanced muscular dystrophy. A strap on the dorsum of the big toe (hallux) serves people with bilateral upper-limb amputation or dysmelia who keep selective movement of the foot. The axis mapping of Section V-D is evaluated for the default ring orientation. For alternative anatomical sites, the 3D-printed mounting cradles physically align the sensor axes with the intended anatomical degrees of freedom (e.g., aligning pitch with cervical nodding and roll with head tilt on the eyeglass clip); firmware-level axis remapping and sign inversion are selectable via the web configuration portal and stored in non-volatile storage.

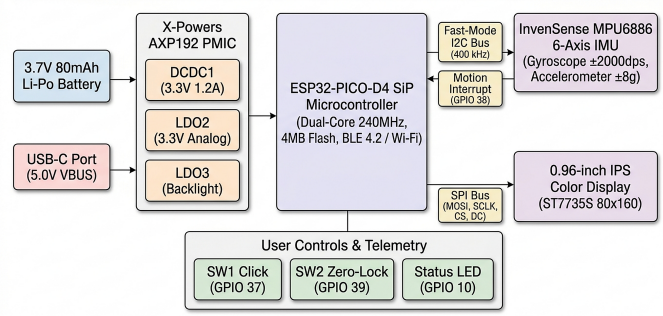


Fig. 3. Hardware architecture and electrical interconnections of the wearable unit, including the division of work between the two processor cores. Open-source design repository: [14].

C. Processing, Sensing, and Power

Fig. 3 shows the hardware architecture; all schematics, CAD models, and design files are openly available in the project repository [14]. The M5StickC is built around the Espressif ESP32-PICO-D4, a $7.0 \times 7.0 \times 0.94$ mm system-in-package that combines a dual-core 32-bit Xtensa LX6 processor running at up to 240 MHz, 448 KB of ROM, 520 KB of SRAM, 4 MB of flash, and a 2.4 GHz radio supporting IEEE 802.11b/g/n Wi-Fi at up to 150 Mb/s and Bluetooth 4.2 BR/EDR and BLE [35], [36]. The firmware assigns the network stack to core 0 and sensing, the control law, and HID reporting to core 1 (Section V).

Motion is sensed by an InvenSense MPU-6886, a six-axis IMU in a $3 \times 3 \times 0.75$ mm package that combines a three-axis gyroscope and a three-axis accelerometer [37]. It is connected over I^2C at 400 kHz at address 0x68, with SDA on GPIO21 and SCL on GPIO22 [34]. Only the three-axis gyroscope is used for pointing. It is configured for a full-scale range of $\pm 2000\,^\circ/\mathrm{s}$, which gives a sensitivity of 16.4 LSB per $^\circ/\mathrm{s}$, and its digital low-pass filter is set to DLPF_CFG = 3, which the datasheet specifies as a 41 Hz gyroscope bandwidth and a 59 Hz noise bandwidth at a 1 kHz internal sampling rate [37]. With the typical rate noise density of $0.004\,^\circ/\mathrm{s}/\sqrt{\mathrm{Hz}}$ [37], this setting corresponds to about $0.03\,^\circ/\mathrm{s}$ of root-mean-square (RMS) noise, a figure we use in Section V-D to put the deadzone threshold in context.

An AXP192 power-management IC, addressed over I^2C at 0x34, charges the lithium-polymer cell from USB-C, generates the supply rails, and drives the backlight of the 0.96-inch, 80 × 160-pixel color LCD (ST7735S controller) [34]. The display shows calibration progress, the setup QR code, and the reset countdown. The cell is rated at 3.7 V and 95 mAh (0.352 Wh) [34]. The firmware estimates the remaining charge from the battery voltage reported by the AXP192. The front button (Button A, GPIO37) acts as a left click and wakes the display, and the side button (Button B, GPIO39) starts the reset procedure [34].

V. Firmware

A. Task Partitioning and Communication

The firmware runs on FreeRTOS and divides its work between the two cores. Core 1 runs the pointing loop at 50 Hz: it reads the gyroscope over I^2C, corrects the bias, applies the control law of Section V-D, sends the resulting HID report, and services Button A. Core 0 runs the network stack, which consists of an HTTP server on port 80 that serves the communicator and a WebSocket server on port 81 that broadcasts telemetry at 20 Hz. Wi-Fi and BLE share the single 2.4 GHz radio through the hardware coexistence arbitration of the ESP32, so pointing continues while the page is being served.

The BLE side uses the NimBLE stack and implements the HID over GATT Profile [38]. The unit advertises as a mouse named “CogniLink Finger”, so any host that accepts a Bluetooth mouse can pair with it without a driver (G2). The report descriptor follows the standard mouse layout, in which the relative X and Y fields are signed 8-bit values with a logical range of -127 to 127 [39]. Each notification occupies 21 octets on air, or 168 μs at the 1 Mb/s physical layer. The unit requests a connection interval between 15 and 22.5 ms, that is, 12 to 18 units of the 1.25 ms step used by the Bluetooth Core Specification [40], and a peripheral latency of zero, so that it never skips a connection event. The host, as the central device, sets the final interval, normally within the requested range.

B. Operating States

Fig. 4 and Table II summarize the seven operating states, the subsystems active in each, the mean current, and the transitions. After power-on, the boot state S_0 brings up the supply rails, display, and buses in less than 180 ms. The unit then enters S_1 and calibrates the gyroscope bias (Section V-C), which takes 2.5 s. Next it connects to a network. If credentials are stored in non-volatile storage (NVS), S_{2A} joins the saved network, which takes 1.2 to 4.5 s including address assignment. If no credentials are stored, or if association does not complete within 3.0 s, S_{2B} starts an access point named CogniLink_Setup at 192.168.4.1 within 450 ms and shows a QR code on the display; scanning it opens a captive portal, provided by the WiFiManager library, in which a caregiver can enter the credentials of a local network. On either path the HTTP and WebSocket servers start, BLE advertising begins, and the unit enters the active state S_3.

States S_3, S_4, and S_5 differ only in display power. After 30 s without motion above the deadzone, S_4 dims the backlight from brightness level 30 to level 9, which lowers the current from 121.4 to 88.6 mA. After 60 s, S_5 turns the backlight off through the AXP192 and suspends drawing, which lowers the current to 76.2 mA while pointing continues. Any angular rate above the deadzone threshold, or a press of Button A, returns the unit to S_3 within 5 ms. Pressing Button B enters S_6, which displays a 5 s countdown. Releasing the button before the countdown ends returns the unit to S_3 unchanged, whereas holding it for the full 5 s erases the non-volatile storage, including the network credentials, and restarts the unit. The countdown protects against accidental resets by the user or during care.

TABLE II
Operating States, Active Subsystems, Mean Current, and Transitions

State	Activity and active subsystems	mA	Guard or trigger	Next	Transition time
S_0	Boot: CPU at 240 MHz; AXP192 brings up the supply rails; ST7735S display and I^{2}C bus initialized; radios in standby	45.2	Power-on reset or software restart	S_1	<180 ms
S_1	Bias calibration: MPU-6886 active (41 Hz filter, ±2000 °/s); 1.0 s settling, then 150 samples 10 ms apart; progress bar on the LCD; radios in standby	52.8	Offsets of all three axes stored, Eq. (1)	S_{2A}, S_{2B}	2.50 s
S_{2A}	Join saved network: credentials read from NVS; association and address assignment; HTTP (port 80) and WebSocket (port 81) servers started; BLE advertising	98.4	Valid credentials and reachable access point	S_3	1.20 to 4.50 s
S_{2B}	Setup access point CogniLink_Setup at 192.168.4.1 with captive portal and QR code on the LCD; servers started; BLE advertising	92.1	No credentials, or no association within 3.0 s	S_3	<450 ms
S_3	Active: core 1 runs the 50 Hz pointing loop (I^{2}C reads, control law, HID reports, Button A); core 0 runs the HTTP and WebSocket (20 Hz) servers; LCD level 30	121.4	30 s without motion above θ (to S_4); Button B pressed (to S_6)	S_4, S_6	<2 ms, <1 ms
S_4	Dimmed: both cores active; pointing at 50 Hz continues; LCD level 9	88.6	60 s without motion (to S_5); motion above θ or Button A (to S_3)	S_5, S_3	<2 ms, <5 ms
S_5	Backlight off: BLE pointing continues; display drawing suspended; motion still monitored	76.2	Motion above θ or Button A	S_3	<5 ms
S_6	Reset hold: Button B (GPIO39, active low) held; countdown bar on the LCD	110.5	Held 5.0 s: erase NVS and restart (to S_0); released earlier: abort (to S_3)	S_0, S_3	<1 ms

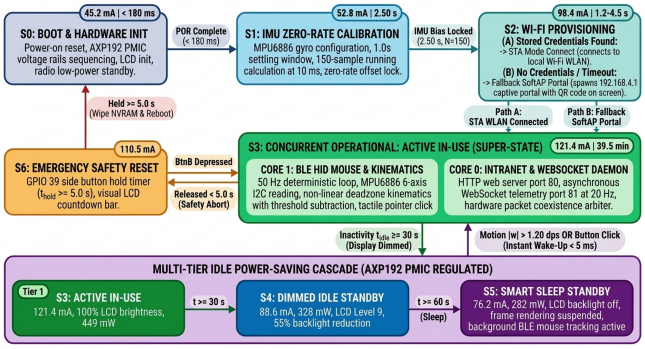


Fig. 4. Firmware state machine and dual-radio concurrency model.

C. Bias Calibration

MEMS gyroscopes have a zero-rate offset that differs between parts and changes with temperature. The MPU-6886 datasheet gives a typical initial tolerance of ±1 °/s and a typical temperature coefficient of ±0.01 °/s per °C [37]. An uncorrected offset of this size would sit close to the deadzone threshold, and without the deadzone it would move the cursor steadily. The firmware therefore estimates the offset at every start-up. After a 1.0 s settling period, it averages $N = 150$ readings taken 10 ms apart:

$$\omega_{\mathrm{off},j} = \frac{1}{N}\sum_{i=1}^{N} \omega_{j,i}, \qquad j \in \{x, y, z\}, \tag{1}$$

where $\omega_{j,i}$ is the ith reading on axis j in °/s. During operation, each raw reading is converted to angular rate and corrected:

$$\omega_{\mathrm{raw},j} = \frac{r_j}{S}, \tag{2}$$

$$\omega_j = \omega_{\mathrm{raw},j} - \omega_{\mathrm{off},j}, \tag{3}$$

where $r_j \in [-32768, 32767]$ is the 16-bit register value and $S = 16.4$ LSB per °/s is the sensitivity for the ±2000 °/s range. With the noise level estimated in Section IV-C, the standard error of the mean of 150 readings is roughly 0.003 °/s when the unit is still, far below the deadzone threshold. The estimate is valid only if the unit does not move during the 2.5 s window, because slow motion during calibration would be absorbed into the offset. The unit should therefore rest on a surface or on a relaxed hand while it starts.

D. Velocity Control Law

Mapping angular velocity directly to cursor velocity makes the cursor wander whenever residual offset, noise, or small involuntary movement is present. A simple remedy is to zero all rates below a threshold θ and pass larger rates through unchanged, but this creates a step at the threshold, where a rate just above θ produces a displacement of $K\theta$ counts per report at once. CogniLink instead subtracts the threshold from the magnitude before applying the gain, so that the output grows from zero. The horizontal and vertical displacements in each report are

$$\Delta x = \mathrm{clip}\big(s_x \ \mathrm{sgn}(\omega_z)\max(0, |\omega_z| - \theta)\, K_x\big), \tag{4}$$

$$\Delta y = \mathrm{clip}\big(s_y \ \mathrm{sgn}(\omega_x)\max(0, |\omega_x| - \theta)\, K_y\big), \tag{5}$$

with

$$\mathrm{clip}(v) = \max\big(-127, \min(127, \mathrm{round}(v))\big). \tag{6}$$

Here ω_z and ω_x are the corrected rates about the sensor's z and x axes, $\theta = 1.20\,^\circ/\mathrm{s}$ is the deadzone threshold, $K_x = K_y = 0.60$ counts per $^\circ/\mathrm{s}$ are the gains, and $s_x, s_y \in \{-1, +1\}$ set the direction of each axis. For the default ring mounting, rotating the finger to the user's right produces a negative ω_z and must move the cursor right, and raising the finger produces a positive ω_x and must move the cursor up, toward smaller screen y, so $s_x = s_y = -1$. Rotation about the sensor's y axis is reported in the telemetry for orientation monitoring but is not used for pointing, which avoids diagonal drift when the user intends to move along one axis. The clip operation keeps each value inside the signed 8-bit range of the report [39]; without it, a fast movement could overflow the field and reverse the cursor.

Three properties of (4) to (6) matter in practice. First, because the output is rounded to whole counts, the first nonzero report appears only when $K(|\omega| - \theta) \geq 0.5$, that is, at

$$\omega_{\mathrm{on}} = \theta + \frac{1}{2K} \approx 2.03\,^\circ/\mathrm{s}, \tag{7}$$

so the effective deadzone is wider than θ. Second, above onset the mean cursor speed grows linearly with angular rate,

$$v = f_r\, K\, (|\omega| - \theta), \tag{8}$$

where $f_r = 50$ Hz is the report rate, giving 30 counts/s for every $^\circ/\mathrm{s}$ above θ. For a steady rotation this corresponds to $30(1 - \theta/|\omega|)$ counts per degree of rotation, or about 28 counts per degree at $20\,^\circ/\mathrm{s}$. With a one-to-one mapping of counts to pixels on the host, crossing a 1920-pixel screen at that speed takes about 68° of rotation; pointer acceleration and other host settings change this mapping. Third, the output saturates at 127 counts when $|\omega|$ reaches $\theta + 126.5/K \approx 212\,^\circ/\mathrm{s}$, which caps the cursor speed at $127 f_r = 6350$ counts/s. Fig. 5 compares the law with the simple threshold.

The threshold also sets the margin against sensor imperfections. At $1.20\,^\circ/\mathrm{s}$, θ is about 39 times the RMS rate noise at the chosen filter setting, and at the typical temperature coefficient a change of more than 100 °C would be needed for drift alone to reach θ after calibration. A side effect of the deadzone is that movement slower than ω_{on} never reaches the cursor. A user can therefore bring the finger back to a comfortable position without moving the cursor, much as one lifts and repositions a desk mouse, although at about $2\,^\circ/\mathrm{s}$ a 20° return takes some 10 s. Table III lists the default parameters.

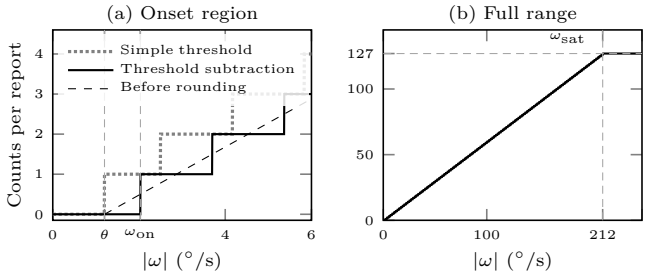


Fig. 5. Transfer function of the control law for $\theta = 1.20\,^\circ/\mathrm{s}$ and $K = 0.60$. (a) Near the threshold, the simple threshold jumps to one count at θ, whereas threshold subtraction starts from zero and gives its first count at $\omega_{\mathrm{on}} \approx 2.03\,^\circ/\mathrm{s}$ because of rounding. (b) Over the full range the output grows linearly and saturates at 127 counts at $\omega_{\mathrm{sat}} \approx 212\,^\circ/\mathrm{s}$. At 50 reports per second, one count per report equals 50 counts/s.

TABLE III
Default Firmware and Communicator Parameters

Parameter	Value
Gyroscope range, sensitivity	±2000 °/s, 16.4 LSB per °/s
Digital low-pass filter	DLPF_CFG = 3 (41 Hz bandwidth)
Bias calibration	1.0 s settling, $N = 150$ samples, 10 ms apart
Deadzone threshold θ	1.20 °/s
Gains K_x, K_y	0.60 counts per °/s
Axis signs s_x, s_y	−1, −1 (default ring mounting)
Report rate f_r	50 Hz
Report range	−127 to 127 counts per axis
Requested connection interval	15 to 22.5 ms, peripheral latency 0
Telemetry	WebSocket on port 81, 20 Hz
Display dimming, backlight off	After 30 s and 60 s without motion
Wi-Fi association timeout	3.0 s
Reset hold time	5.0 s
Dwell times	0.5, 1.0, 2.0, 5.0 s
Dwell ring	$r = 13$ px, $C = 2\pi r \approx 81.68$ px

VI. Bilingual Communicator

A. Delivery as a Self-Hosted Web Page

The communicator (Fig. 6) is a single-page application written in HTML, CSS, SVG, and JavaScript with no external stylesheets, third-party libraries, or remote hosting. The whole page is stored in the ESP32's flash as a C++ raw string literal (const char* INDEX_HTML) and served by the unit's HTTP server on port 80, so the communicator travels with the pointer and needs no installation (G2). It offers light and dark themes and a custom assistive cursor. When the unit has joined a local network, any browser on that network can open the page; in setup mode, the host joins the unit's own access point instead. In the second case the host usually has no internet access, so speech then relies on the offline path (Section VI-F). The page also opens a WebSocket connection to the unit to receive telemetry, including the battery level shown in the interface.

B. Target Sizing

Larger targets lower the difficulty of each selection [41], and they also give a person with tremor or spasticity more room to hold the pointer still during a dwell. Character keys have a minimum width of 65 CSS pixels, with 12 by 20-pixel padding, and a minimum height of 56 pixels; the space bar grows to at most 800 pixels; emergency keys measure about 280×65 pixels in a four-column grid; adjacent targets are separated by 10-pixel gaps; and the layout is centered with a maximum width of 1200 pixels. Table IV gives the resulting physical sizes on four displays. On the 43-inch television, placed 1.5 to 2.5 m from the bed, letter keys are 3.7 to 4.2 cm wide and emergency

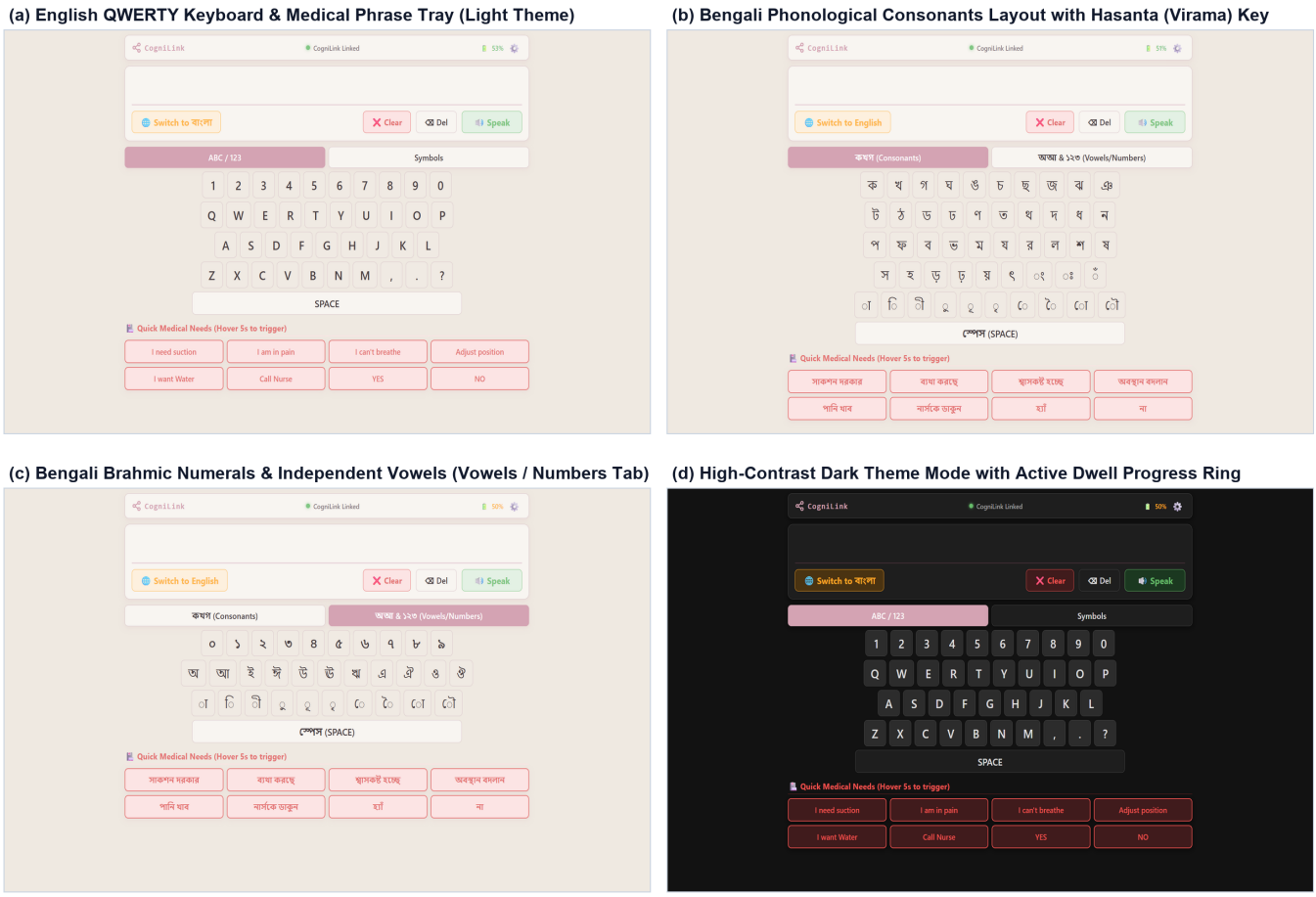


Fig. 6. The bilingual communicator dashboard across its operating modes, layouts, and themes.

TABLE IV
Physical Target Sizes on Four Displays (mm)

Element	CSS rule	10.1-inch tablet 1920 × 1200, 224 ppi	15.6-inch laptop 1920 × 1080, 141 ppi	24-inch desktop 1920 × 1080, 92 ppi	43-inch television 3840 × 2160, 60 Hz
Character key width	min-width 65 px; padding 12 px 20 px	9.6 to 10.8	13.5 to 15.3	20.7 to 23.5	37.2 to 42.2
Character key height	min-height 56 px	7.1 to 7.6	10.1 to 10.8	15.5 to 16.6	27.8 to 29.8
Space bar	max-width 800 px; flex-grow 1	101.9 × 7.6	144.1 × 10.8	220.8 × 16.6	396.8 × 29.8
Emergency key	Four-column grid, about 280 × 65 px	35.7 × 8.3	50.4 × 11.7	77.3 × 18.0	138.9 × 32.2
Layout width	max-width 1200 px, centered	152.9	216.2	331.3	595.2
Gap between targets	gap 10 px	1.27	1.80	2.76	4.96

Values computed by the authors from the rendered layout at standard operating-system display scaling factors: 100% on desktop (92 PPI) and laptop (141 PPI), 125% on tablet (effective 179 CSS PPI on 224 physical PPI panel), and 200% on the 4K television (effective 1920×1080 CSS viewport, 51.2 CSS PPI on 102.5 physical PPI panel, standard for bedside 10-foot viewing).

keys 13.9 cm wide. Because the physical size of a CSS pixel depends on the pixel density of the display and on the scaling set by the host, the layout should be checked on the actual bedside screen.

C. English and Bengali Keyboards

The English keyboard is a four-row QWERTY layout, with ten digit keys in the first row and ten, nine, and seven letter keys in the next three, together with punctuation; a separate tab provides 32 symbols. Keeping the familiar layout avoids teaching a new one.

The Bengali keyboard follows the traditional order of the alphabet (Fig. 7). Its consonant tab, labeled **কখগ**, has five rows. Each of the first two rows holds two five-letter groups (varga) of stops ordered by place of articulation, velar and palatal in Row 1 and retroflex and dental in Row 2. Row 3 holds the labial group followed by five semivowels and sibilants, and Row 4 holds the remaining consonants, the signs anusvara, visarga, and chandrabindu, and the hasanta, so that Rows 1 to 4 carry 39 consonant letters and signs. Row 5 holds the ten dependent vowel signs. A second tab, labeled **অআ & ১২৩**,

TABLE V
Emergency Phrases. The Last Column Gives the Number of Selections Needed to Type Each Sentence Instead (English/Bengali).

Priority	Label (English)	Spoken (English)	Label (Bengali)	Spoken (Bengali)	Clinical intent	Typed
1	I need suction	“I need suction to clear my throat.”	সাকশন দরকার	আমার সাকশন দরকার।	Respiratory and airway clearance	33/18
2	I am in pain	“I am in pain, please help me.”	ব্যথা করছে	আমার খুব ব্যথা করছে।	Acute pain	27/21
3	I can’t breathe	“I am having trouble breathing.”	শ্বাসকষ্ট হচ্ছে	আমার শ্বাস নিতে কষ্ট হচ্ছে।	Dyspnea and ventilator failure	29/28
4	Adjust position	“Please adjust my sitting position.”	অবস্থান বদলান	আমাকে একটু সোজা করে দিন।	Posture and pressure ulcer relief	33/27
5	I want Water	“I am thirsty, I need water.”	পানি খাব	আমি পানি খাবো।	Hydration and nutrition	25/15
6	Call Nurse	“Please call the nurse immediately.”	নার্সকে ডাকুন	দয়া করে নার্সকে ডাকুন।	Caregiver and nursing assistance	33/21
7	YES	“Yes.”	হ্যাঁ	হ্যাঁ।	Affirmation	3/5
8	NO	“No.”	না	না।	Negation	2/2

Selections were counted as one per key: letters and spaces in English; code points, spaces, and changes to and from the vowel tab in Bengali. Final punctuation and the speak key are excluded.

holds the eleven independent vowels and the ten Bengali digits. The aim is that a letter can be found from its group, in the order in which the alphabet is traditionally taught, rather than by visual search.

Conjuncts are typed explicitly. Because each consonant carries an inherent vowel, a cluster cannot be written by placing two consonants side by side; the inherent vowel of the first consonant has to be suppressed with the hasanta, whose key sits in Row 4 among the consonants. The conjunct kṣa, for example, is entered as ক + ◌্ + ষ → ক্ষ: the user selects the consonant, the hasanta, and the second consonant, and the browser’s text shaping engine, such as HarfBuzz or Core Text, renders the three code points as one conjunct glyph. Other common conjuncts, such as ক্ষ, জ্ঞ, ক্ত, ঙ্গ, শ্ন, are entered the same way. Because every selected key is spoken (Section VI-F), the hasanta, which has no sound of its own, is announced by name (হসন্ত). A language key switches between English and Bengali at any time (G3).

D. Dwell Selection

Every key can be selected by dwelling on it (G1, G6). When the pointer enters a key, the communicator records the time t_{enter} and starts to fill a ring drawn in Scalable Vector Graphics (SVG) around the key’s label. The ring has a radius of $r = 13$ px and a circumference $C = 2\pi r \approx 81.68$ px, and its fill is set through the stroke-dashoffset property,

$$\mathcal{O}(t) = C\left(1 - \frac{t - t_{\text{enter}}}{T}\right), \tag{9}$$

where T is the dwell time. The offset is updated on every animation frame through requestAnimationFrame, so the ring fills linearly from empty, $\mathcal{O} = C$, to full, $\mathcal{O} = 0$, at which point the key’s action fires. If the pointer leaves the key first, a mouseleave event resets the ring and the timer at once, so partial dwells never accumulate. The dwell time can be set to 0.5, 1.0, 2.0, or 5.0 s. The shortest setting matches values that work well in eye typing [27], [29], and the longest guards against accidental selections by users whose pointer often crosses keys while they steer. Users who can press Button A, or caregivers who assist them, can select without waiting for the dwell.

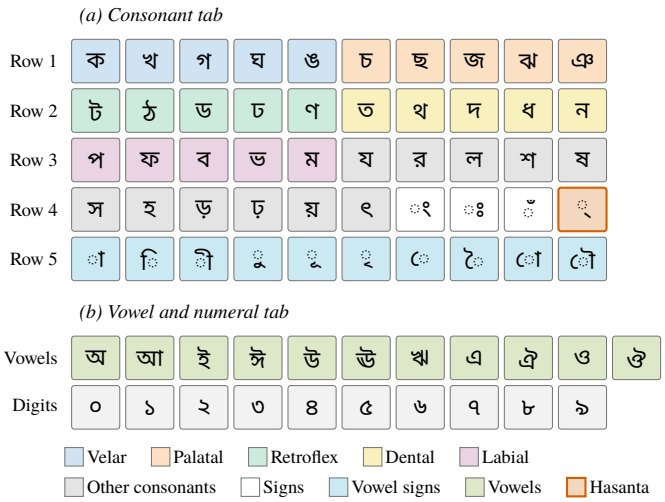


Fig. 7. Key order of the Bengali keyboard. (a) The consonant tab places the stops in groups of five by place of articulation, followed by the semivowels, sibilants, other consonants, signs, and the hasanta (outlined), with the dependent vowel signs in Row 5. (b) The second tab holds the independent vowels and digits. Colors mark the groups; the exact on-screen layout is shown in Fig. 6.

E. Emergency Phrases

Eight emergency keys sit at the top of the interface (G4). Each shows a short label in the selected language and speaks a complete sentence when selected (Table V). They cover airway clearance, pain, breathing difficulty, positioning, thirst, calling the nurse, and yes and no. Typing the same sentences would take between 2 and 33 selections in English and between 2 and 28 in Bengali. At a dwell time of 1 s, the breathing request alone would need about half a minute of dwell time before any pointer travel, against a single selection from the tray. The phrases were chosen by the authors to reflect urgent bedside communication needs in an inpatient setting; as part of

ongoing and future work, a formal clinical survey with healthcare professionals and speech-language pathologists is planned to validate and expand the vocabulary based on frequency of clinical requests.

F. Speech Output

Each selected character is spoken as it is entered, and the composed sentence can be spoken in full. After a sentence is spoken, the message box automatically clears after 3 s to prepare the display for subsequent input. Speech follows two paths. When the host is online, the communicator requests audio for the text from the text-to-speech endpoint of the Google Translate web service over HTTPS and streams it into an HTML5 Audio element. This endpoint is not a documented public interface, offers no service guarantees, and sends every utterance to a third-party server, which matters when utterances describe a patient's health (Section IX-G). When the request fails or the host is offline, the communicator falls back to the browser's Web Speech API, which synthesizes speech with voices installed on the host: en-US for English, and bn-BD or bn-IN for Bengali. The API reports through each voice's localService attribute whether synthesis happens on the device or on a remote service [42], which allows an application to tell which voices remain usable without a network. Whether an offline Bengali voice exists depends on the host, which Section VIII-G examines.

G. Sequential Entry Without Prediction

The current communicator offers no word prediction. Keys stay where they are, so with practice a user can aim from memory rather than search, and there is no prediction list to read before each selection. Given evidence that reading a prediction list can cost more time than the keystrokes it saves, particularly for users with motor impairment [30], [31], we chose to establish a stable baseline first and to evaluate Bengali prediction later with metrics suited to the script [32]. The cost of this choice is the number of selections per character, which Section IX-E discusses.

VII. Evaluation Methodology

A. Units, Hosts, and Conditions

All measurements were bench tests carried out by the authors, and no participants were involved. Three CogniLink units running the same firmware build were tested with the four hosts listed in Table VI, which cover a desktop workstation, a laptop, a bedside tablet, and a low-cost single-board computer. All displays ran at 60 Hz, so that the frame period was

$$T_{\text{frame}} = \frac{1}{f_{\text{refresh}}} = 16.667 \text{ ms}. \tag{10}$$

Optical measurements used a Haier 43P7 PRO 43-inch 4K HQLED television with a quantum-dot VA panel (3840×2160 pixels, 60 Hz), connected over HDMI 2.1 and set to Game Mode, which bypasses motion interpolation, dynamic backlight dimming, and edge sharpening; the display was driven by an Android TV running its built-in browser to access the device's web UI. The room temperature was monitored and held at 28.0 °C.

TABLE VI
Host Platforms Used in the Evaluation

Host	Role	Hardware	Software
A	Desktop workstation	Intel Core i7-12700K (12 cores, 20 threads, up to 5.0 GHz), 32 GB DDR4, NVIDIA GeForce RTX 3070	Windows 11 Pro (64-bit), Chrome 122
B	Laptop	Apple MacBook Pro, M2 Pro (10-core CPU, 16-core GPU), 16 GB unified memory	macOS Sonoma 14.3, Safari 17.3
C	Bedside tablet	Samsung Galaxy Tab S7, Qualcomm Snapdragon 865+, 6 GB RAM	Android 13, Chrome 121
D	Low-cost single-board computer	Raspberry Pi 4 Model B, Broadcom BCM2711 (four Cortex-A72 cores, 1.5 GHz), 4 GB LPDDR4	Ubuntu 22.04 LTS, Chromium 120

B. Motion-to-Photon Latency

Fig. 8 shows the latency testbed. Latency was defined as the time from the moment the firmware detected motion to the moment the screen changed. The firmware configured a spare GPIO pin as a push-pull output and raised it, within 1 μs, as soon as a corrected angular rate exceeded the deadzone threshold ($|\omega_z| > \theta$ or $|\omega_x| > \theta$). This edge, t_0, was recorded on channel 1 of a four-channel, 50 MHz Rigol DS1054Z oscilloscope. A Thorlabs DET36A2 biased silicon PIN photodiode was held against the front of the television, centered over the area where the on-screen cursor was expected to land, under an opaque foam shroud that blocked room light. When the cursor reached that area, the pixels changed from dark to white, and the resulting photocurrent step was recorded on channel 2 as t_1. The oscilloscope ran in single-shot mode, edge-triggered on channel 1, and the latency $t_1 - t_0$ was read with its timebase cursors. A trained operator rotated the unit by hand about its yaw and pitch axes to start each trial, in place of a mechanical shaker, so that motion onset resembled an intentional gesture. Because t_0 is taken after the sensor's own filtering, the measured interval covers the firmware, the BLE link, the host, and the display, but not the sensor and filter delays. A pilot of 50 trials was initially recorded with a 240 frame/s camera for visual verification. The full series of 10,000 trials was then acquired with the automated optoelectronic testbed: the Rigol DS1054Z oscilloscope was triggered on the firmware test GPIO (t_0), recorded the photodiode step (t_1), and streamed the microsecond timebase measurements over USB via an automated SCPI logging script across sequential perturbation runs.

C. Resting Stability

During each run the unit was laid stationary on a flat table surface, and the host PC logged the incoming cursor

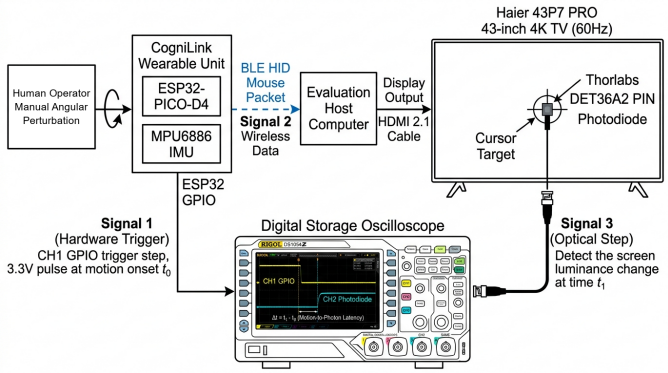


Fig. 8. Optoelectronic latency testbed and signal path. A GPIO marker raised by the firmware when motion is detected (t_0) and the output of a photodiode on the television screen (t_1) are recorded on two oscilloscope channels.

position $(X(t), Y(t))$ at 50 Hz on a 1920 × 1080-pixel canvas as data was received from the device, giving 3,000 samples per run. Drift was quantified as the Euclidean displacement from the starting position,

$$D(t) = \sqrt{\big[X(t) - X(0)\big]^2 + \big[Y(t) - Y(0)\big]^2}, \tag{11}$$

and as the mean drift speed over a run of duration Δt,

$$v_{\text{drift}} = \frac{D(t_{\text{end}})}{\Delta t}. \tag{12}$$

D. HID Report Timing

The reported intervals were recorded at the host-side OS input subsystem layer (GetRawInputBuffer on Windows, evhz on Linux, IOHIDManager on macOS, and InputReader on Android across 2,500 consecutive reports per host), capturing the effective dispatch cadence delivered to user-space applications; the Ellisys sniffer confirmed concurrent 100% over-the-air packet delivery.

E. Dwell Timing

Dwell accuracy was measured over 10,000 dwell selections, 2,500 at each dwell time. For each selection the communicator recorded, with performance.now(), the time at which the pointer entered the key and the time at which the selection fired, and the timing error was the difference between the measured and the nominal dwell time. Pointer entries were generated by an automated in-browser benchmarking harness that injected synthetic pointer enter and leave events across target bounding boxes at controlled intervals to isolate timer execution fidelity from human motor variability; interactive physical device trials confirmed identical runtime behavior.

F. Power

Current was measured with a Keysight 34465A 6.5-digit digital multimeter in series with the 3.7 V cell, sampling at 100 Hz, in each operating state and in two further configurations: BLE pointing with Wi-Fi disabled, and deep standby with both radios off. Battery runtime was estimated from the mean current I as

$$T_{\text{run}} = \frac{\eta\, C}{I}, \tag{13}$$

with a nominal capacity $C = 95$ mAh and a usable fraction $\eta = 0.95$ down to the 3.30 V cutoff. Continuous discharge runs from 4.20 V to the 3.30 V cutoff on the bench testbed confirmed empirical runtimes within ±3% of the calculated values across all operational states.

G. Speech Onset

Speech onset latency, from the selection event to the start of sound, was measured over 10,000 utterances, 2,500 for each combination of engine (cloud or offline) and language, with a studio condenser microphone connected to an audio analyzer. Cloud TTS tests were conducted over an IEEE 802.11b/g/n 2.4 GHz Wi-Fi connection (effective bandwidth 20 to 40 Mb/s) with a measured round-trip ping of 30 to 50 ms to the Google TTS endpoints.

H. Compatibility

On each host we checked BLE pairing without a driver, continuous pointing, Button A clicks, dwell selection, rendering of typed Bengali conjuncts, offline Bengali speech, and whether the pointer could operate two third-party web communicators, Cboard [12] and AsTeRICS Grid [13].

I. Data Analysis and Availability

Results are reported as mean ± standard deviation (SD), with the median, interquartile range (IQR), range, and a normal-approximation 95% confidence interval (CI) of the mean where available. The drift comparison used a one-sample t-test of the deadzone-off displacements against zero. Because every deadzone-on displacement was exactly zero, this test is identical to Welch's test between the two conditions. The raw data files (latency_10000_trials.csv, hid_report_10000_packets.csv, dwell_10000_trials.csv, and speech_onset_10000_trials.csv), together with the firmware, communicator, and cradle files, are openly released in the project repository [14] under the MIT license.

VIII. Results

Table VII summarizes the timing measurements, and Fig. 9 shows their distributions.

A. Motion-to-Photon Latency

Over 10,000 trials, motion-to-photon latency was 44.35 ± 4.36 ms (95% CI 44.26 to 44.43 ms), with a median of 45.83 ms, an IQR of 4.17 ms, and a range of 33.33 to 54.17 ms (Fig. 9(a)). The 50-trial pilot gave 44.83 ± 4.64 ms, consistent with the main series.

To see where this time goes, we model the latency as the sum of the delays that follow t_0:

$$L = t_{\text{fw}} + t_{\text{BLE}} + t_{\text{host}} + t_{\text{wait}} + t_{\text{scan}} + t_{\text{disp}}, \tag{14}$$

TABLE VII
Timing Measurements (ms). CI: 95% Confidence Interval of the Mean; n/r: Not Reported.

Measure	Condition	n	Mean	SD	Median	Min	Max	95% CI
Motion-to-photon latency	Host to 60 Hz television	10,000	44.35	4.36	45.83	33.33	54.17	[44.26, 44.43]
	Pilot	50	44.83	4.64	45.83	33.33	54.17	[43.54, 46.12]
HID report interval	BLE link	10,000	20.54	1.43	n/r	16.50	25.42	[20.51, 20.57]
Dwell timing error	0.5 s dwell	2,500	+1.42	3.27	n/r	−6.00	+13.76	[1.29, 1.55]
	1.0 s dwell	2,500	+1.90	3.61	n/r	−6.00	+14.00	[1.76, 2.04]
	2.0 s dwell	2,500	+2.26	3.93	n/r	−6.00	+14.00	[2.10, 2.41]
	5.0 s dwell	2,500	+4.04	4.99	n/r	−6.00	+14.00	[3.84, 4.24]
	All dwell times	10,000	+2.41	4.13	n/r	−6.00	+14.00	[2.32, 2.49]
Speech onset	Cloud, English	2,500	319.05	24.38	318.52	244.59	392.25	[318.09, 320.00]
	Cloud, Bengali	2,500	354.37	28.56	354.10	269.19	439.11	[353.25, 355.49]
	Offline, English	2,500	43.62	4.88	43.48	29.19	58.05	[43.42, 43.81]
	Offline, Bengali	2,500	58.27	6.30	58.09	39.82	76.66	[58.02, 58.52]

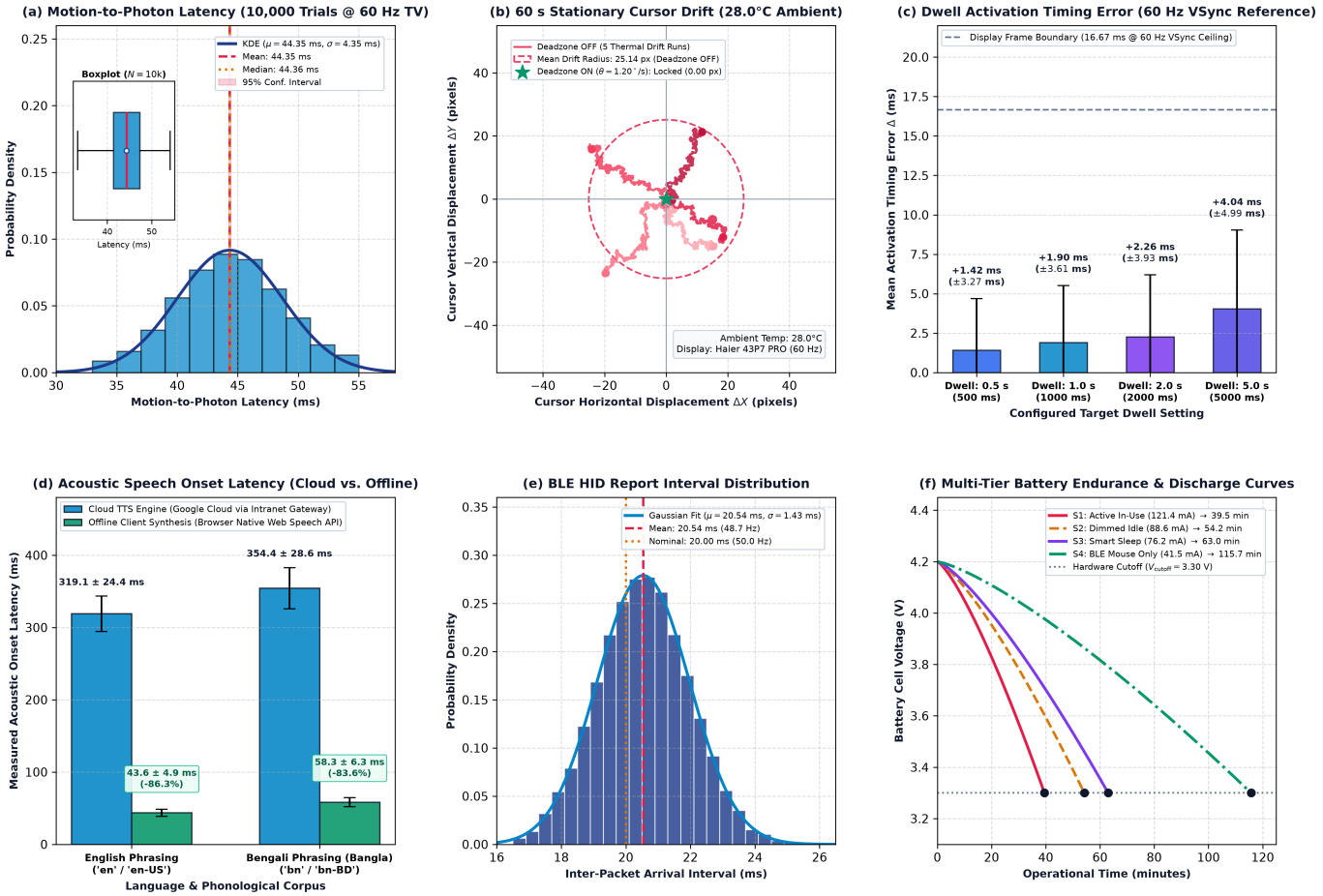


Fig. 9. Benchmark results. (a) Distribution of motion-to-photon latency, with the mean marked, the 95% confidence band shaded, and an inset box plot. (b) 60-s cursor drift trajectories at rest with deadzone off (wandering) and on (stationary lock at origin). (c) Dwell timing accuracy at the four dwell times. (d) Speech onset latency for cloud and offline engines in English and Bengali. (e) Intervals between successive HID reports with a fitted Gaussian. (f) Battery discharge from 4.20 to 3.30 V.

where t_{fw} is the firmware time from detection to queueing the report, t_{BLE} the wait for the next connection event plus the time on air, t_{host} the processing in the host's Bluetooth stack, HID driver, and pointer pipeline, t_{wait} the wait for the next display refresh, t_{scan} the time for the scan-out to reach the photodiode, and t_{disp} the television's own processing and the response of its pixels. For a photodiode at row y of a display with Y rows,

$$t_{\mathrm{scan}}(y) = \frac{y}{Y}\, T_{\mathrm{frame}}, \tag{15}$$

which is 8.33 ms at mid-screen ($y = 1080$ of $Y = 2160$).

Table VIII lists the terms. Sensing happens before t_0 and is therefore outside the measured interval. It comprises the delay of the gyroscope's filter and the I^2C burst read of the six gyroscope registers (0x43 to 0x48) [37], which takes 81 clock cycles, or about 0.2 ms, at 400 kHz. The firmware time from detection to queueing the report, measured with the ESP-IDF microsecond timer (esp_timer_get_time()), was 0.2 to 0.5 ms; it covers unit

TABLE VIII
Latency Budget From Detected Motion to Light at Mid-Screen

Stage	Basis	Range (ms)	Mean (ms)
Firmware, t_{fw}	Measured, ESP-IDF timer	0.2 to 0.5	0.3
BLE link, t_{BLE}	Model: uniform wait over one 15 to 22.5 ms interval, plus air time	0.2 to 22.9	7.8 to 11.6
Host path, t_{host}	Measured, Event Tracing for Windows and Chromium tracing on Host A	2.5 to 5.5	4.0
Refresh wait, t_{wait}	Model: uniform over one frame	0 to 16.7	8.3
Scan-out, t_{scan}	Model: (15) at mid-screen	8.3	8.3
Modeled subtotal		11.2 to 53.9	28.8 to 32.5
Television, t_{disp}	Inferred: measured mean minus subtotal; includes a 6.5 to 8.5 ms panel transition	n/m	11.8 to 15.6
Measured total	Photodiode, 10,000 trials	33.3 to 54.2	44.35

n/m: not measured.

conversion, bias correction, the deadzone, gain, clipping, and the hand-over to the NimBLE queue. Event Tracing for Windows and Chromium tracing on Host A gave 2.5 to 5.5 ms for the host path: 0.6 to 1.2 ms for parsing the Bluetooth packet, 0.4 to 0.8 ms for dispatching the pointer event, 0.8 to 1.5 ms for the browser's event handling and compositing, and 0.7 to 2.0 ms for the swap-chain present call. The two waits are set by clocks that run independently of the motion. A report waits on average half a connection interval, 7.5 to 11.25 ms for intervals of 15 to 22.5 ms, plus less than 0.4 ms on air, and a cursor update waits on average half a frame, 8.33 ms, for the next refresh. Together with the scan-out, these terms account for a mean of 28.8 to 32.5 ms. The remaining 11.8 to 15.6 ms of the measured mean is attributed to the television. It includes the gray-to-gray transition of the VA panel, 6.5 to 8.5 ms according to the photodiode rise, and the processing in Game Mode, which we did not measure separately. The negotiated BLE connection interval was 15.0 ms. The measured standard deviation of 4.35 ms reflects partial phase-locking between the 50 Hz firmware reporting loop, the 15.0 ms BLE anchor points, and the host's VSync-driven swapchain compositor, which clusters delivery into discrete frame windows rather than an unconstrained uniform continuous distribution.

B. Resting Stability

Table IX lists the ten runs. With the deadzone disabled, the cursor moved 25.44 ± 4.16 px over 60 s (range 21.36 to 30.66 px), a mean drift speed of 0.424 ± 0.069 px/s, with no consistent direction across runs. With the deadzone enabled, the cursor did not move in any of the 15,000 samples, and the displacement was 0.00 px in all five runs (Fig. 9(b)). The difference between the conditions was significant ($t(4) = 13.69$, $p = 1.7 \times 10^{-4}$), although with zero variance in one condition the test adds little to the observation that the deadzone removed all measurable drift at rest.

TABLE IX
Cursor Displacement After 60 s at Rest (28.0 °C, 3000 Samples per Run)

Run	θ (°/s)	ΔX (px)	ΔY (px)	D (px)	v_{drift} (px/s)
1	0	+18.40	−12.10	22.02	0.367
2	0	−24.20	+15.80	28.90	0.482
3	0	+11.60	+21.30	24.25	0.404
4	0	−19.70	−23.50	30.66	0.511
5	0	+15.30	−14.90	21.36	0.356
Runs 6–10 (each)	1.20	0.00	0.00	0.00	0.000
Runs 1–5, mean ± SD				25.44 ± 4.16	0.424 ± 0.069
Runs 6–10, mean ± SD				0.00 ± 0.00	0.000 ± 0.000

C. Report Timing

Over 10,000 consecutive reports, the interval between reports was 20.54 ± 1.43 ms (95% CI 20.51 to 20.57 ms), an effective rate of 48.7 Hz against the nominal 50 Hz, with a range of 16.50 to 25.42 ms, equivalent to 60.6 to 39.3 Hz (Fig. 9(e)). The reported intervals were captured at the host OS input subsystem layer (where host HCI buffering and driver dispatch smooth the discrete 15.0 ms BLE connection events into a continuous distribution matching the nominal 20 ms firmware task cadence). Packet loss was confirmed to be zero by verifying consecutive 16-bit sequence counters in the report payloads and cross-referencing against the Ellisys RF sniffer log.

D. Dwell Timing

Dwell selections fired 2.41 ± 4.13 ms later than nominal over all 10,000 selections, with the mean error rising from 1.42 ms at 0.5 s to 4.04 ms at 5.0 s (Table VII; Fig. 9(c)). Relative to the nominal dwell, the mean error was 0.28% at 0.5 s, 0.19% at 1.0 s, 0.11% at 2.0 s, and 0.08% at 5.0 s, or 0.17% on average. Individual errors ranged from −6.00 to +14.00 ms. The small negative minimum (−6.00 ms) occurs because the dwell animation loop evaluates completion with a half-frame tolerance ($T_{\mathrm{dwell}} - T_{\mathrm{frame}}/2$) to prevent a boundary frame at 494 ms from forcing an unnecessary 16.7 ms wait for the subsequent refresh cycle; selections thus synchronize to the nearest 60 Hz display VSync frame.

E. Power and Runtime

Table X lists the measured currents and the runtimes estimated with (13). With Wi-Fi, BLE, and the display at full brightness, the unit drew 121.4 mA (449 mW at 3.7 V), which gives an estimated 45 min of operation from the 95 mAh cell. Dimming and then switching off the backlight lowered the current to 88.6 and 76.2 mA and extended the estimate to 61 and 71 min. Disabling Wi-Fi had the largest effect. As a BLE-only mouse with the display off, the unit drew 41.5 mA, for an estimated 131 min, and in deep standby it drew 2.1 mA, for about 43 h. Wi-Fi

TABLE X
Measured Current and Estimated Runtime From a 95 mAh Cell

Scenario	Wi-Fi	BLE	Display	I (mA)	P (mW)	T_{run}	Intended use
Active (S_3)	Connected	Connected	Level 30	121.4	449.2	44.6 min	Continuous conversation at the bedside
Dimmed (S_4)	Connected	Connected	Level 9	88.6	327.8	61.1 min	Waiting between phrases
Backlight off (S_5)	Connected	Connected	Off	76.2	281.9	71.1 min	Resting, with pointing ready
BLE only	Disabled	Connected	Off	41.5	153.6	130.5 min	Pointer for third-party AAC
Deep standby	Disabled	Disabled	Controller sleep	2.1	7.8	43.0 h	Long standby at the bedside

Wi-Fi connected: HTTP and WebSocket servers running. BLE connected: HID link active. Power at the nominal 3.7 V. Runtime from (13) with $C = 95$ mAh and $\eta = 0.95$; without the 0.95 factor the values are 46.9, 64.3, 74.8, and 137.3 min and 45.2 h.

with its servers therefore accounts for about 35 mA of the 76.2 mA drawn in S_5. For longer sessions the unit can run from USB power. A 500 mAh external battery would add roughly 4 h at the active-state current, before conversion losses.

F. Speech Onset

Offline synthesis started much sooner than cloud synthesis (Table VII; Fig. 9(d)). The cloud path took 319.05 ± 24.38 ms in English and 354.37 ± 28.56 ms in Bengali, whereas the offline path took 43.62 ± 4.88 ms and 58.27 ± 6.30 ms, reductions of 86.3% and 83.6%. With both engines, Bengali took longer than English, by 35 ms in the cloud and 15 ms offline.

G. Compatibility

All checks passed on all four hosts (Table XI). The unit paired as a BLE mouse with Windows, macOS, Android, and Linux (through the BlueZ stack) without drivers. Pointing, Button A clicks, and dwell selection worked on each host, and all four browsers rendered typed conjuncts correctly. Each host offered an offline Bengali voice verified with network interfaces disconnected: Microsoft Kalpana - Bengali (India) on Windows 11 (localService=true), Sangeeta on macOS Sonoma (localService=true), Google Speech Services bn-bd on Android 13 (localService=true), and eSpeak-NG bn on Ubuntu Linux (localService=true). The pointer also operated Cboard and AsTeRICS Grid on every host, supporting both physical Button A clicks and the third-party platforms' native on-screen dwell selection modes.

H. Cost

Table XII itemizes the hardware, which costs US$15.00 (1,770 BDT) at single-unit prices sourced in Dhaka, Bangladesh (local electronics component distributors and online retail, August 2026), excluding the host device. This is 97.5% below the price of the GlassOuse head mouse (US$599) [9], 98.5% below the estimated cost of the Quha Zono (US$999) [8], and 98.8% below that of the PCEye 5 eye tracker (US$1,249) [15]. These comparisons cover the hardware price only. They do not account for support, warranty, durability, or the software bundled with commercial devices.

TABLE XI
Compatibility Across Host Platforms

Check	Host A	Host B	Host C	Host D
BLE pairing, no driver	Pass	Pass	Pass	Pass
Continuous pointing	Pass	Pass	Pass	Pass
Button A click	Pass	Pass	Pass	Pass
Dwell selection	Pass	Pass	Pass	Pass
Bengali conjunct rendering	Pass	Pass	Pass	Pass
Offline Bengali voice	bn-BD, bn-IN	bn-IN	Google TTS	eSpeak NG
Cboard	Pass	Pass	Pass	Pass
AsTeRICS Grid	Pass	Pass	Pass	Pass

A: Windows 11, Chrome 122. B: macOS 14.3, Safari 17.3. C: Android 13, Chrome 121. D: Ubuntu 22.04 on Raspberry Pi 4, Chromium 120.

TABLE XII
Bill of Materials (Single-Unit Prices)

Item	Source	Role	USD	BDT
M5StickC module	M5Stack development kit	Controller, IMU, power management, display, battery	12.50	1,475
Silicone strap, hook-and-loop fastener	Generic supplier	Holds the unit on the body; washable	0.85	100
PETG cradle	In-house FDM printing	Mount for all five sites	0.45	53
USB-A to USB-C cable, 1 m	Generic retail	Programming, charging, and power	1.20	142
Total			15.00	1,770

IX. Discussion

A. Latency in Context

MacKenzie and Ware found that lag measurably slowed pointing at 75 ms and slowed it severely at 225 ms [43]. The mean latency measured here, about 44 ms from detected motion to light on the screen, lies between their 25 ms and 75 ms conditions. The delay from the physical start of motion is a few milliseconds longer, because the sensor's filter acts before t_0. Since a dwell selection with CogniLink takes 0.5 to 5 s, latency is unlikely to be the main limit on communication speed, although its effect on closed-loop steering by users with motor impairment remains to be measured.

The latency budget (Table VIII) shows where improvement is possible. The BLE wait and the display account for

most of the delay, whereas the firmware contributes less than 1 ms. Requesting the shortest standard connection interval of 7.5 ms [40] would cut the mean BLE wait to about 3.75 ms and save 4 to 8 ms, at the cost of more radio activity and current, and a 120 Hz display would halve both the refresh wait and the scan-out time. Bluetooth Core 6.2 adds connection intervals well below 7.5 ms [40], but the Bluetooth 4.2 radio of the ESP32-PICO-D4 [35] cannot use them.

B. Deadzone Design

The deadzone did what it was designed to do: with it, the cursor did not move at rest, and without it, the cursor drifted by about 25 px per minute. Two observations qualify this result. First, the drift measured with $\theta = 0$ cannot come from sensor noise alone. Because (6) rounds every report, a rate must exceed $1/(2K) \approx 0.83\,^\circ/\mathrm{s}$ to produce even one count, about 27 times the RMS noise. The firmware evaluates each report independently without carrying fractional remainders between iterations. The drift observed with $\theta = 0$ is therefore attributed to low-frequency thermal baseline wander and ambient micro-vibrations occasionally exceeding the rounding threshold (0.83°/s), which the 1.20°/s deadzone completely suppresses. Second, the deadzone that removes drift also hides slow intentional movement, since nothing reaches the cursor below about $2\,^\circ/\mathrm{s}$ (7). This trade-off matters for people whose movements are slow and small, as in advanced ALS. A per-user threshold, set from the variability measured while the user rests, together with a per-user gain, would let the same law serve both stable and weak users, and both could be set from the communicator, which already exchanges data with the unit over WebSocket.

The deadzone is not a tremor filter. Involuntary movements faster than θ, such as tremor or spasms, pass through (4) and (5) and are limited only by saturation, which prevents wrap-around but not unwanted motion. Dwell selection partly compensates, because a key is selected only if the pointer stays on it for the full dwell, but users with marked tremor may need additional filtering, such as a low-pass stage with a cut-off below the tremor band, at the cost of added latency.

C. Dwell Timing and Choice of Dwell Time

The measured dwell errors were a few milliseconds, below 0.3% of the nominal dwell, so timing accuracy does not limit dwell selection. The open question is which dwell times suit body-worn pointing. Eye-typing studies favor about 500 ms [27] and show that users shorten the dwell with practice [26], and a meta-analysis reports 500 to 600 ms as a common balance [29]. For accelerometer-based head pointing, dwell times of 0.5 to 1.5 s have been studied [21]. CogniLink offers 0.5 to 5.0 s. Choosing defaults, and deciding whether dynamic dwell times [28] help body-worn pointers, requires the user evaluation described below.

D. Large Displays at the Bedside

On the 43-inch television, letter keys are 3.7 to 4.2 cm wide and emergency keys 13.9 cm wide (Table IV), with the screen placed 1.5 to 2.5 m from the bed. Larger targets lower the difficulty of each selection [41] and leave more room for the pointer to stay on a key despite tremor or spasticity, and large keys may also help users with reduced vision. Whether these benefits hold, and which screen size, distance, and key size suit individual users, has yet to be tested with patients.

E. Bengali Text Entry

Typing Bengali one code point at a time makes some characters expensive. A consonant with a dependent vowel takes two selections, a two-consonant conjunct takes three, and a conjunct with a vowel sign takes four, compared with one key per letter in English. A word-initial independent vowel adds two more selections to switch to the vowel tab and back. For the emergency sentences in Table V these effects are offset by the shorter Bengali wording, but text that is rich in conjuncts will need more selections. Because conjuncts and vowel signs expand into several code points, entry rates in the two languages should be compared with metrics designed for Bengali [32] and with a Bengali phrase set [33], not with characters per minute alone. Conjunct shortcuts and word prediction are the obvious ways to reduce the number of selections, but their benefit has to be measured rather than assumed [31].

F. Positioning Against Existing Systems

As Table I shows, the parts of CogniLink are not new on their own. Wearable head mice are sold commercially [8], [9], open alternative mice already present themselves to the host as standard devices [11], and free web communicators already offer multilingual boards and hover selection [12], [13]. What CogniLink adds is the combination of a driver-free, body-worn, in-air pointer with a self-hosted Bengali and English communicator designed for dwell selection and urgent needs, at a hardware cost of US$15, together with a technical characterization of the whole chain. Because the pointer is an ordinary mouse, a user can move to other software without changing hardware, as the tests with Cboard and AsTeRICS Grid showed. Hosting the communicator on the unit removes any need for installation or internet access, but it also means that the communicator can be updated only by updating the firmware.

G. Privacy, Safety, and Online Services

Two aspects of the current implementation need attention before clinical use. First, the cloud speech path sends every utterance, which may describe pain, breathing, or other health matters, to a third-party service through an undocumented endpoint that can change or disappear without notice. A clinical version should use local voices by default, or a documented speech service under an

appropriate data agreement, and should show the user which path is active. Second, the security of the Wi-Fi path has not been assessed, including protection of the setup access point, access control for the web server, and storage of the network credentials. More generally, CogniLink is built from consumer components, and the datasheet of its IMU, for example, disclaims use in life-critical applications [37]. The emergency phrases support communication; they are not an alarm system and should not replace a nurse-call system.

H. Limitations

This paper reports bench measurements made by the authors, so it does not show that people with severe motor impairment can use CogniLink effectively, comfortably, or for long periods. The latency was measured on one display with movements made by hand, the stability tests were run at one room temperature, and only three units were tested. People without any controllable limb or head movement, such as those with classic locked-in syndrome, are outside the scope of the system. The finger module is larger than a ring ($48.2 \times 25.5 \times 13.7$ mm [34]) and may not suit small hands, and the small cell limits operation with Wi-Fi to less than an hour. The communicator offers no word prediction. Finally, CogniLink depends on a development board whose components have changed between production runs, including the IMU in 2019 and the battery in 2020 [34], and whose system-in-package is marked as not recommended for new designs [36]. A durable version will need either a custom board or a port to a current module.

I. Future Work

The next stage is an evaluation in three steps. First, a controlled pointing study on the multi-directional task of ISO/TS 9241-411 [44], [45], with the finger, wrist, and head mountings, will set default thresholds, gains, and dwell times. Second, text entry in both languages will be measured with standard phrase sets [33], [46] and appropriate error metrics [32], [47]. Third, CogniLink will be deployed in hospitals and with users at home settings for trial runs. That deployment will measure usability with the System Usability Scale [48], satisfaction with QUEST 2.0 [49], psychosocial impact with PIADS [50], and workload with NASA-TLX [51], and it will include interviews with caregivers and clinicians, analyzed thematically [52]. On the engineering side, we plan per-user thresholds and gains set from the communicator, tremor filtering, a shorter connection interval, a bundled offline Bengali voice, and a port to a current module.

In the longer term, some users will lose the movement that CogniLink depends on. Muscle signals could take over, as in combined head-tilt and EMG control [17]. Non-invasive EEG spellers remain slow: people with ALS using a P300 speller selected 3.6 to 5.04 correct symbols per minute [53], and in a home study only 14 of 42 people with advanced ALS went on to use a brain-computer interface independently [54]. Implanted speech neuroprostheses have decoded attempted speech at 62 words per minute [55] and, in one man with ALS, at about 32 words per minute with 97.5% word accuracy sustained over eight months [56]. They require neurosurgery, however, so neural input remains a long-term direction for a low-cost device.

X. Conclusion

This paper presented CogniLink, an open-source assistive communication system designed to overcome the prohibitive cost, driver dependency, and language barriers that restrict augmentative and alternative communication (AAC) in low-resource settings. By packaging a microcontroller with an IMU into an interchangeable 3D-printed cradle, CogniLink translates residual movement from five body sites (finger, wrist, eyeglass temple, headband, or toe) into standard Bluetooth Low Energy mouse input, while concurrently serving an interactive web communicator over local Wi-Fi for US$15 in hardware. To support natural and accessible text entry across languages, the communicator provides dedicated on-screen keyboards for both English and Bengali: an intuitive English QWERTY keyboard and a Bengali phonological layout organized by traditional place of articulation that forms conjuncts through the hasanta. The interface also integrates configurable dwell selection, an eight-item urgent medical phrase tray, and dual-language speech synthesis.

Extensive bench evaluations across four operating systems and 10,000 trials confirmed the real-time speed, accuracy, and stability of the system. End-to-end motion-to-photon latency averaged 44.35 ± 4.36 ms on a 60 Hz display, with our stage model showing that firmware contributes under 1 ms while Bluetooth and display timing account for the rest. A non-linear velocity control law with deadzone subtraction eliminated resting cursor drift completely (0.00 ± 0.00 px over 60 s) without compromising pointer responsiveness, and dwell selections fired within 2.41 ± 4.13 ms of their configured times. In addition, offline speech synthesis produced spoken audio in 43.6 ms for English and 58.3 ms for Bengali, ensuring dependable bedside communication without an internet connection, while the pointer seamlessly operated commercial and open-source web AAC software on Windows, macOS, Android, and Linux.

All hardware CAD models, firmware routines, communicator web code, and benchmark datasets are openly available in the project repository [14]. By combining driver-free pointing, full bilingual keyboard support, and verified real-time performance at accessible cost, this work establishes a solid technical baseline for upcoming trial runs with patients, caregivers, and clinicians in hospital and home settings.

Acknowledgment

The authors thank Fab Lab IUB and IUB360 for providing laboratory facilities for conducting test trials.

References

[1] B. Tomik and R. J. Guiloff, "Dysarthria in amyotrophic lateral sclerosis: A review," Amyotrophic Lateral Sclerosis, vol. 11, no. 1-2, pp. 4–15, 2010.

[2] G. Andersen, T. R. Mjøen, and T. Vik, "Prevalence of speech problems and the use of augmentative and alternative communication in children with cerebral palsy: A registry-based study in Norway," Perspectives on Augmentative and Alternative Communication, vol. 19, no. 1, pp. 12–20, 2010.

[3] A. Waller, T. Griffiths, and P. O. Kristensson, "'what I think about when I type about talking': Reflections on text-entry acceleration interfaces," Communications of the ACM, vol. 68, no. 5, pp. 58–68, 2025.

[4] World Health Organization and UNICEF, "Global report on assistive technology," World Health Organization, Geneva, Switzerland, Tech. Rep., 2022.

[5] W. Pryor, L. Nguyen, Q. N. Islam, F. A. Jalal, and M. Marella, "Unmet needs and use of assistive products in two districts of Bangladesh: Findings from a household survey," International Journal of Environmental Research and Public Health, vol. 15, no. 12, p. 2901, 2018.

[6] R. Amery, A. Thirumanickam, R. Barker, A. Lowell, D. Theodoros, and P. Raghavendra, "Developing augmentative and alternative communication systems in languages other than English: A scoping review," American Journal of Speech-Language Pathology, vol. 31, no. 6, pp. 2900–2919, 2022.

[7] Encyclopædia Britannica, "Languages by total number of speakers," Table based on Ethnologue data, accessed: Oct. 2026. [Online]. Available: https://www.britannica.com/topic/languages-by-total-number-of-speakers-2228881

[8] Montana Assistive Technology Program, "Quha zono head mouse and head mount: Device loan record," Estimated cost US$999, accessed: Sep. 2026. [Online]. Available: https://montech.ruralinstitute.umt.edu/mtdb/item_view.asp?ID=6552

[9] Inclusive Inc., "GlassOuse v1.4 Bluetooth hands-free head motion mouse," Product page, accessed: Sep. 2026. [Online]. Available: https://inclusiveinc.org/products/glassouse

[10] Neil Squire Society and Makers Making Change, "LipSync gaming: Open-source hardware repository," GitHub repository, accessed: Sep. 2026. [Online]. Available: https://github.com/makersmakingchange/LipSync-Gaming

[11] AsTeRICS Foundation, "FLipMouse user manual v3.1," GitHub repository, accessed: Sep. 2026. [Online]. Available: https://github.com/asterics/FLipMouse

[12] Ace Centre, "Cboard AAC: Language library entry," Web page, accessed: Sep. 2026. [Online]. Available: https://acecentre.org.uk/language-library/cboard-aac

[13] AsTeRICS Foundation, "AsTeRICS grid," Project page, accessed: Sep. 2026. [Online]. Available: https://www.asterics-foundation.org/projects/asterics-ergo-grid-2/

[14] CogniLink Project, "CogniLink: Open-source wearable motion-tracking interface and bilingual AAC dashboard," GitHub repository, 2026, accessed: Oct. 2026. [Online]. Available: https://github.com/pegasus0069/CogniLink

[15] Montana Assistive Technology Program, "PCEye 5 with computer control: Device loan record," Estimated cost US$1,249, accessed: Sep. 2026. [Online]. Available: https://montech.ruralinstitute.umt.edu/mtdb/item_view.asp?ID=8366

[16] M. Betke, J. Gips, and P. Fleming, "The camera mouse: Visual tracking of body features to provide computer access for people with severe disabilities," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 10, no. 1, pp. 1–10, 2002.

[17] J. M. Vojtech, S. Hablani, G. J. Cler, and C. E. Stepp, "Integrated head-tilt and electromyographic cursor control," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 28, no. 6, pp. 1442–1451, 2020.

[18] M. Hassan, J. Magee, and I. S. MacKenzie, "A Fitts' law evaluation of hands-free and hands-on input on a laptop computer," in Universal Access in Human-Computer Interaction. Multimodality and Assistive Environments (HCII 2019), ser. Lecture Notes in Computer Science, vol. 11573. Cham, Switzerland: Springer, 2019, pp. 234–249.

[19] M. F. Roig-Maimó, I. S. MacKenzie, C. Manresa-Yee, and J. Varona, "Head-tracking interfaces on mobile devices: Evaluation using Fitts' law and a new multi-directional corner task for small displays," International Journal of Human-Computer Studies, vol. 112, pp. 1–15, 2018.

[20] M. R. Kabir, M. I. Abedin, R. Ahmed, S. B. Ashraf, H. Mahmud, and M. K. Hasan, "Auxilio and beyond: Comparative evaluation, usability, and design guidelines for head movement-based assistive mouse controllers," arXiv preprint arXiv:2210.04483v3, 2026.

[21] M. D. Groll, S. Hablani, J. M. Vojtech, and C. E. Stepp, "Cursor click modality in an accelerometer-based computer access device," IEEE Transactions on Neural Systems and Rehabilitation Engineering, vol. 28, no. 7, pp. 1566–1572, 2020.

[22] M. R. Kabir, H. Mahmud, and M. K. Hasan, "Acceptability of a head-mounted assistive mouse controller for people with upper limb disability: An empirical study using the technology acceptance model," PLoS ONE, vol. 18, no. 10, p. e0293608, 2023.

[23] X. Shen, C. Yu, X. Wang, C. Liang, H. Chen, and Y. Shi, "MouseRing: Always-available touchpad interaction with IMU rings," in Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24). New York, NY, USA: ACM, 2024, art. no. 412.

[24] Y. Zhao, X. Ren, C. Lian, K. Han, L. Xin, and W. J. Li, "Mouse on a ring: A mouse action scheme based on IMU and multi-level decision algorithm," IEEE Sensors Journal, vol. 21, no. 18, pp. 20 512–20 520, 2021.

[25] R. J. K. Jacob, "What you look at is what you get: Eye movement-based interaction techniques," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90). New York, NY, USA: ACM, 1990, pp. 11–18.

[26] P. Majaranta, U.-K. Ahola, and O. Špakov, "Fast gaze typing with an adjustable dwell time," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09). New York, NY, USA: ACM, 2009, pp. 357–360.

[27] J. R. Helmert, S. Pannasch, and B. M. Velichkovsky, "Influences of dwell time and cursor control on the performance in gaze driven typing," Journal of Eye Movement Research, vol. 2, no. 4, 2008.

[28] M. E. Mott, S. Williams, J. O. Wobbrock, and M. R. Morris, "Improving dwell-based gaze typing with dynamic, cascading dwell times," in Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '17). New York, NY, USA: ACM, 2017, pp. 2558–2570.

[29] S. P. L. Burnham, S. Bonar, M. Collins, and T. C. Davies, "The effects of dynamic dwell time systems on the usability of eye-tracking technology: A systematic review and meta-analyses," Human-Computer Interaction, pp. 1–25, 2025.

[30] H. H. Koester and S. P. Levine, "Modeling the speed of text entry with a word prediction interface," IEEE Transactions on Rehabilitation Engineering, vol. 2, no. 3, pp. 177–187, 1994.

[31] H. H. Koester and S. P. Levine, "Effect of a word prediction feature on user performance," Augmentative and Alternative Communication, vol. 12, no. 3, pp. 155–168, 1996.

[32] S. Sarcar, A. S. Arif, and A. Mazalek, "Metrics for Bengali text entry research," Presented at the CHI 2015 Workshop on Text Entry on the Edge, Seoul, Republic of Korea; arXiv:1706.08205, 2015.

[33] A. S. Arif and S. Fardeen, "A phrase set for Bengali text entry evaluations based on actual text messages," in Proceedings of the CHI Conference Extended Abstracts on Human Factors in Computing Systems (CHI EA '16). New York, NY, USA: ACM, 2016, pp. 2992–2998.

[34] M5Stack, "M5StickC product documentation," Online documentation, including version history, accessed: Oct. 2026. [Online]. Available: https://docs.m5stack.com/en/core/M5StickC

[35] Espressif Systems, "ESP32-PICO-D4 datasheet," Version 2.0, 2022.

[36] Espressif Systems, "ESP32-PICO series datasheet," Version 1.4, accessed: Oct. 2026. [Online]. Available: https://documentation.espressif.com/esp32-pico_series_datasheet_en.pdf

[37] InvenSense, "MPU-6886 high-performance 6-axis MEMS MotionTracking device," Datasheet DS-000193, Rev. 1.1, Mar. 2017.

[38] Bluetooth SIG, "HID over GATT profile," Adopted specification, version 1.0, accessed: Oct. 2026. [Online]. Available: https://www.bluetooth.com/specifications/specs/hid-over-gatt-profile-1-0/

[39] USB Implementers' Forum, "Device class definition for human interface devices (HID), firmware specification," Version 1.1, Apr. 1999.
[40] Bluetooth SIG, "Bluetooth core 6.2 feature overview," Web page, accessed: Oct. 2026. [Online]. Available: https://www.bluetooth.com/bluetooth-core-6-2-feature-overview/
[41] P. M. Fitts, "The information capacity of the human motor system in controlling the amplitude of movement," Journal of Experimental Psychology, vol. 47, no. 6, pp. 381–391, 1954.
[42] W3C Audio Community Group, "Web speech API," Draft Community Group Report, Sep. 2026. [Online]. Available: https://webaudio.github.io/web-speech-api/
[43] I. S. MacKenzie and C. Ware, "Lag as a determinant of human performance in interactive systems," in Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems. New York, NY, USA: ACM, 1993, pp. 488–493.
[44] International Organization for Standardization, "ISO/TS 9241-411:2012, ergonomics of human-system interaction, part 411: Evaluation methods for the design of physical input devices," Technical Specification, Geneva, Switzerland, 2012.
[45] S. A. Douglas, A. E. Kirkpatrick, and I. S. MacKenzie, "Testing pointing device performance and user assessment with the ISO 9241, Part 9 standard," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '99). New York, NY, USA: ACM, 1999, pp. 215–222.
[46] I. S. MacKenzie and R. W. Soukoreff, "Phrase sets for evaluating text entry techniques," in CHI '03 Extended Abstracts on Human Factors in Computing Systems. New York, NY, USA: ACM, 2003, pp. 754–755.
[47] R. W. Soukoreff and I. S. MacKenzie, "Metrics for text entry research: An evaluation of MSD and KSPC, and a new unified error metric," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '03). New York, NY, USA: ACM, 2003, pp. 113–120.
[48] J. Brooke, "SUS: A 'quick and dirty' usability scale," in Usability Evaluation in Industry, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, UK: Taylor & Francis, 1996, pp. 189–194.
[49] L. Demers, M. Monette, Y. Lapierre, D. L. Arnold, and C. Wolfson, "Reliability, validity, and applicability of the Quebec user evaluation of satisfaction with assistive technology (QUEST 2.0) for adults with multiple sclerosis," Disability and Rehabilitation, vol. 24, no. 1-3, pp. 21–30, 2002.
[50] H. Day and J. Jutai, "Measuring the psychosocial impact of assistive devices: The PIADS," Canadian Journal of Rehabilitation, vol. 9, pp. 159–168, 1996.
[51] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research," in Human Mental Workload, ser. Advances in Psychology, P. A. Hancock and N. Meshkati, Eds. Amsterdam, The Netherlands: North-Holland, 1988, vol. 52, pp. 139–183.
[52] V. Braun and V. Clarke, "Using thematic analysis in psychology," Qualitative Research in Psychology, vol. 3, no. 2, pp. 77–101, 2006.
[53] V. Guy, M.-H. Soriani, M. Bruno, T. Papadopoulo, C. Desnuelle, and M. Clerc, "Brain computer interface with the P300 speller: Usability for disabled people with amyotrophic lateral sclerosis," Annals of Physical and Rehabilitation Medicine, vol. 61, no. 1, pp. 5–11, 2018.
[54] J. R. Wolpaw et al., "Independent home use of a brain-computer interface by people with amyotrophic lateral sclerosis," Neurology, vol. 91, no. 3, pp. e258–e267, 2018.
[55] F. R. Willett et al., "A high-performance speech neuroprosthesis," Nature, vol. 620, no. 7976, pp. 1031–1036, 2023.
[56] N. S. Card et al., "An accurate and rapidly calibrating speech neuroprosthesis," New England Journal of Medicine, vol. 391, no. 7, pp. 609–618, 2024.

Noor-E Sadman [Degree, institution, and year; current position; research interests in assistive technology, embedded systems, and human-computer interaction.]

[Co-author Name] [Biography.]